شرح سطر بسطر ٩ — الإقلاع: إلى أين نذهب عند فتح التطبيق؟

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

عند فتح التطبيق يجب أن يعرف إلى أين يذهب دون أن يفكّر الطالب. هذان الملفان هما «دماغ الإقلاع»:

• لا توجد ملفات ← الترحيب ثم إنشاء الملف.

• ملف واحد بلا رمز سري ← الرئيسية مباشرة.

• أكثر من ملف، أو ملف عليه رمز سري ← «من يتعلّم الآن؟».

القرار نفسه دالة صرفة (`decideLaunchPlan`) فتُختبر بلا واجهة، وشاشة الإقلاع تنفّذه فقط.

lib/features/launch/launch_plan.dart — قرار الإقلاع

lib/features/launch/launch_plan.dart

الأسطر 1–4

```
1  import 'package:flutter_riverpod/flutter_riverpod.dart';
2
3  import '../../core/db/app_database.dart';
4  import '../profiles/profile_providers.dart';
```

ماذا يفعل؟ استيراد Riverpod والقاعدة (لنوع `Profile`) ومزوّدات الملفات.

لماذا بهذه الطريقة؟ نقرأ الملفات من المستودع عبر المزوّد.

الأسطر 6–15

```
6  enum LaunchTarget {
7    /// لا توجد ملفات: ابدأ من الترحيب وإنشاء الملف.
8    welcome,
9
10   /// ملف واحد بلا رمز سري: ادخل الرئيسية مباشرة.
11   home,
12
13   /// أكثر من ملف أو ملف محمي برمز سري: اختر من يتعلّم.
14   picker,
15 }
```

ماذا يفعل؟ تعداد `LaunchTarget` بثلاث وجهات: ترحيب ورئيسية واختيار ملف، مع تعليق لكل منها.

لماذا بهذه الطريقة؟ ثلاث نتائج فقط تكفي لحالات الإقلاع كلها.

الأسطر 17–24

```
class LaunchPlan {
  const LaunchPlan(this.target, {this.profileId});

  final LaunchTarget target;

  /// الملف الذي يُفتح مباشرة عند `home`.
  final String? profileId;
}
```

ماذا يفعل؟ صنف `LaunchPlan`: الوجهة ومعها معرّف الملف عند `home`.

لماذا بهذه الطريقة؟ نحتاج معرّف الملف لأن الرئيسية تفتح ملفًا بعينه، بينما الوجهتان الأخريان لا تحتاجانه.

الأسطر 26–30

```
/// قواعد التنقل عند فتح التطبيق (دالة صرفة لتسهل اختبارها):
/// 1. لا ملفات ← الترحيب.
/// 2. ملف واحد بلا رمز سري ← الرئيسية.
/// 3. غير ذلك ← «من يتعلّم الآن؟».
LaunchPlan decideLaunchPlan(List<Profile> profiles) {
```

ماذا يفعل؟ تعليق القواعد وتوقيع الدالة الصرفة `decideLaunchPlan(profiles)`.

لماذا بهذه الطريقة؟ تأخذ قائمة الملفات وتعيد خطة بلا أي اعتماد على Flutter أو القاعدة فتُختبر بسهولة (4 اختبارات).

السطر 31

```
  if (profiles.isEmpty) return const LaunchPlan(LaunchTarget.welcome);
```

ماذا يفعل؟ القاعدة 1: لا ملفات ← الترحيب.

لماذا بهذه الطريقة؟ الجهاز نظيف: يبدأ الطالب من الصفر.

الأسطر 32–34

```
  if (profiles.length == 1 && profiles.single.pinHash == null) {
    return LaunchPlan(LaunchTarget.home, profileId: profiles.single.id);
  }
```

ماذا يفعل؟ القاعدة 2: ملف واحد بلا رمز سري (`pinHash == null`) ← الرئيسية بذلك الملف.

لماذا بهذه الطريقة؟ لا داعي لسؤال «من يتعلّم؟» إن كان هناك طالب واحد غير محمي.

الأسطر 35–36

```
  return const LaunchPlan(LaunchTarget.picker);
}
```

ماذا يفعل؟ القاعدة 3: غير ذلك ← اختيار الملف، ثم إغلاق الدالة.

لماذا بهذه الطريقة؟ أكثر من ملف أو ملف محمي: يلزم اختيار (وربما رمز).

الأسطر 38–42

```dart
/// يُعاد حسابها كل مرة تُفتح فيها شاشة الإقلاع (autoDispose).
final launchPlanProvider = FutureProvider.autoDispose<LaunchPlan>((ref) async {
  final profiles = await ref.read(profileRepositoryProvider).getAll();
  return decideLaunchPlan(profiles);
});
```

ماذا يفعل؟ `FutureProvider.autoDispose` : `launchPlanProvider` يقرأ كل الملفات ويستدعي القرار.

لماذا بهذه الطريقة؟ `autoDispose` يُعيد حسابه كلما فُتحت شاشة الإقلاع من جديد، فلا يبقى قرار قديم. نستعمل `read` وليس `watch` لأنه يُحسب مرة واحدة.

lib/features/launch/launch_screen.dart — شاشة الإقلاع الصامتة

lib/features/launch/launch_screen.dart

الأسطر 1–3

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material وRiverpod وgo_router.

لماذا بهذه الطريقة؟ الشاشة تقرأ مزوّدًا وتتنقّل.

الأسطر 5–8

```dart
import '../../app/routes.dart';
import '../../core/widgets/mascot.dart';
import '../profiles/active_profile_provider.dart';
import 'launch_plan.dart';
```

ماذا يفعل؟ استيراد المسارات والشخصية ومزوّد الملف النشط وخطة الإقلاع.

لماذا بهذه الطريقة؟ كل ما تحتاجه.

الأسطر 10–16

```dart
/// شاشة إقلاع صامتة: تقرأ الملفات من القاعدة ثم تنتقل إلى الوجهة الصحيحة.
class LaunchScreen extends ConsumerStatefulWidget {
  const LaunchScreen({super.key});

  @override
  ConsumerState<LaunchScreen> createState() => _LaunchScreenState();
}
```

ماذا يفعل؟ تعليق وصنف `LaunchScreen` من نوع `ConsumerStatefulWidget` .

لماذا بهذه الطريقة؟ نحتاج `initState` لبدء القرار مرة واحدة عند الظهور.

الأسطر 18–23

```
class _LaunchScreenState extends ConsumerState<LaunchScreen> {
  @override
  void initState() {
    super.initState();
    _route();
  }
```

ماذا يفعل؟ الحالة: في `initState` نستدعي `_route()` بعد `super.initState()`.

لماذا بهذه الطريقة؟ يبدأ القرار فور فتح الشاشة.

الأسطر 25–28

```
  Future<void> _route() async {
    final LaunchPlan plan;
    try {
      plan = await ref.read(launchPlanProvider.future);
```

ماذا يفعل؟ `_route`: نعلن متغيّر الخطة ثم نقرأها بـ `await` داخل `try`.

لماذا بهذه الطريقة؟ قراءة الملفات من القاعدة غير متزامنة.

الأسطر 29–32

```
    } catch (_) {
      // تعذّر قراءة القاعدة: نبدأ من الترحيب بدل شاشة عالقة.
      if (mounted) context.go(AppRoutes.welcome);
      return;
```

ماذا يفعل؟ إن فشلت القراءة نذهب للترحيب (إن كانت الشاشة قائمة) ونخرج.

لماذا بهذه الطريقة؟ بدل شاشة عالقة بلا نهاية؛ أسوأ حالة أن يبدأ الطالب من الترحيب.

الأسطر 33–34

```
    }
    if (!mounted) return;
```

ماذا يفعل؟ نغلق `catch` ونتأكد أن الشاشة ما زالت قائمة.

لماذا بهذه الطريقة؟ لا ننتقل إن غادر المستخدم أثناء الانتظار (`mounted`).

الأسطر 36–38

```
    switch (plan.target) {
      case LaunchTarget.welcome:
        context.go(AppRoutes.welcome);
```

ماذا يفعل؟ `switch` على الوجهة: `context.go(welcome)` ← `welcome`.

لماذا بهذه الطريقة؟ `go` وليس `push` فلا يرجع الطالب إلى شاشة الإقلاع بزر الرجوع.

الأسطر 39–40

```
      case LaunchTarget.picker:
        context.go(AppRoutes.profiles);
```

ماذا يفعل؟ `go(profiles)` ← `picker`.

لماذا بهذه الطريقة؟ شاشة اختيار الملف.

الأسطر 41–45

```
      case LaunchTarget.home:
        await ref
            .read(activeProfileIdProvider.notifier)
            .select(plan.profileId!);
        if (mounted) context.go(AppRoutes.home);
```

ماذا يفعل؟ `home`: نجعل الملف المقصود نشطًا ثم نذهب للرئيسية.

لماذا بهذه الطريقة؟ الاختيار أولًا لأن الرئيسية تقرأ الملف النشط؛ ثم نفحص `mounted` مجددًا بعد `await`.

الأسطر 46–47

```
    }
  }
```

ماذا يفعل؟ إغلاق `switch` والدالة.

لماذا بهذه الطريقة؟ نهاية `_route`.

الأسطر 49–53

```
  @override
  Widget build(BuildContext context) {
    return const Scaffold(body: Center(child: Mascot(size: 96)));
  }
}
```

ماذا يفعل؟ `build`: شاشة بها الشخصية بحجم 96 في المنتصف ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ شاشة مؤقتة قصيرة جدًا؛ الشخصية تعطي إحساسًا بتطبيق حي بدل شاشة بيضاء.